03 - Text Baseline Models

Objective

In this notebook, we establish the first baseline models for the Rakuten product-classification task:

1. **Largest-class baseline:** A minimal floor score predicting the most common category.
2. **TF-IDF + Linear SVM:** A fast, standard text classifier.
3. **Visual interpretability:** Inspecting what product text and images look like for key categories.
4. **Cleaning comparison:** An experiment testing whether text cleaning actually improves the baseline.
5. **Error analysis:** Inspecting where the classifier succeeds, where it gets confused, and why.

1. Load the Prepared Text

We load the preprocessed dataset with the shared 80/20 train/validation split. The `text_combined` column contains product title and description merged together.

```
Training products:   67,932
Validation products: 16,984
```

Analysis

- We have **67,932 training products (80%)** and **16,984 validation products (20%)**.
- The split is stratified by product category, so all 27 categories are represented in both sets in the exact same proportions.
- Keeping this split fixed ensures fair comparison across all future models (text, image, and multimodal).

2. Largest-Class Baseline

This baseline predicts the most common training category for every product. It ignores the text completely and sets the absolute minimum score that any useful model must beat.

```
Largest class: 2583
Weighted F1:   0.0258
Macro F1:      0.0080
```

Analysis

- Always guessing the most frequent category (class 2583) yields a **Weighted F1 of 0.0258** and a **Macro F1 of 0.0080**.
- Because the catalog has 27 categories, naive guessing gets almost everything wrong.
- This sets our absolute floor: any trained machine learning model must beat this score easily.

3. Convert Text into TF-IDF Features

We convert the text into numerical TF-IDF features using word unigrams and bigrams up to 50,000 features. The vocabulary is learned only from the training set to avoid data leakage.

```
Training matrix:   (67932, 50000)
Validation matrix: (16984, 50000)
```

Analysis

- Each product is now represented as a sparse vector across **50,000 word and 2-word n-gram features**.
- TF-IDF gives higher weight to terms specific to certain products (like *playstation* or *sommier*) and downweights common French words (like *pour* or *avec*).
- Fitting only on the training set protects us from data leakage.

4. Train the First Text Classifier (Linear SVM)

Linear SVM is a standard, fast baseline for high-dimensional TF-IDF text features. It trains in seconds and provides a strong reference score for later models.

	model	weighted_f1	macro_f1
0	Largest-class baseline	0.0258	0.0080
1	TF-IDF + Linear SVM	0.8274	0.8156

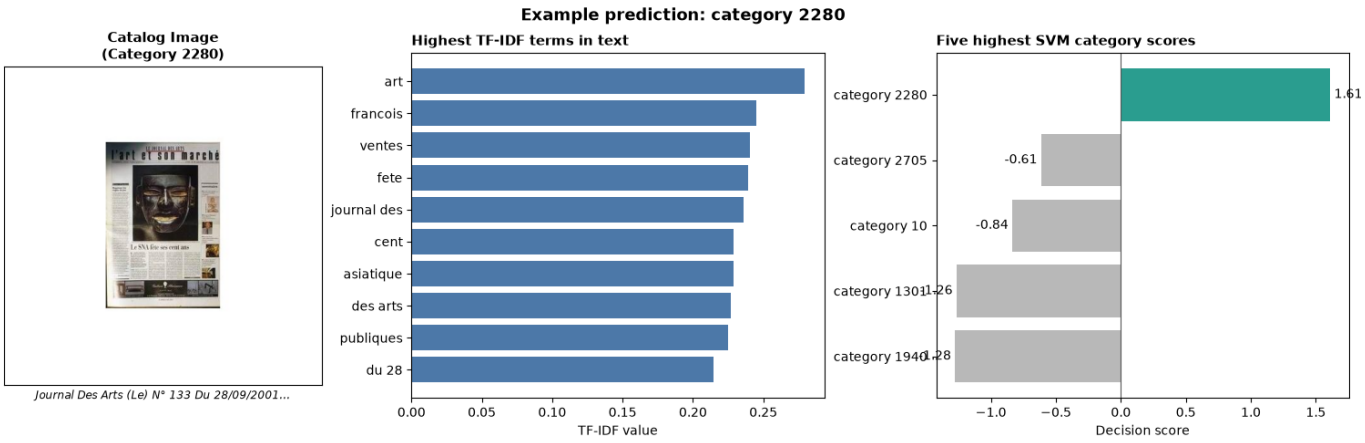
Analysis

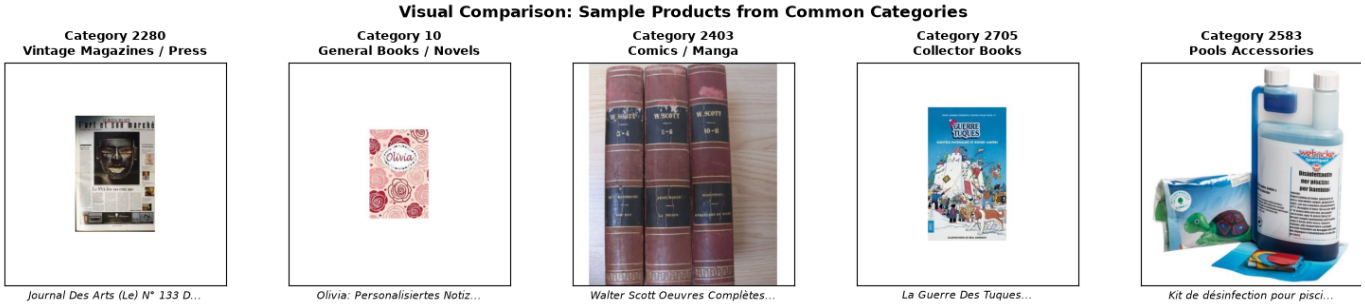
- The Linear SVM achieves a **Weighted F1 of 0.8274**, comfortably beating the official Rakuten text benchmark of **0.8113**.
- **Macro F1 is 0.8156**, very close to the weighted score, showing that the model performs well across both large and small categories without neglecting minority classes.
- The model trains in under 20 seconds, making it a very strong and fast reference point.

5. How the Classifier Makes Decisions & What Categories Look Like

To see what the model is actually learning, we take a correctly predicted product and inspect its **catalog image**, its **top keywords**, and the model's **decision scores**.

We also look at sample product images from common categories to see what these numeric category codes actually represent in reality.





Analysis

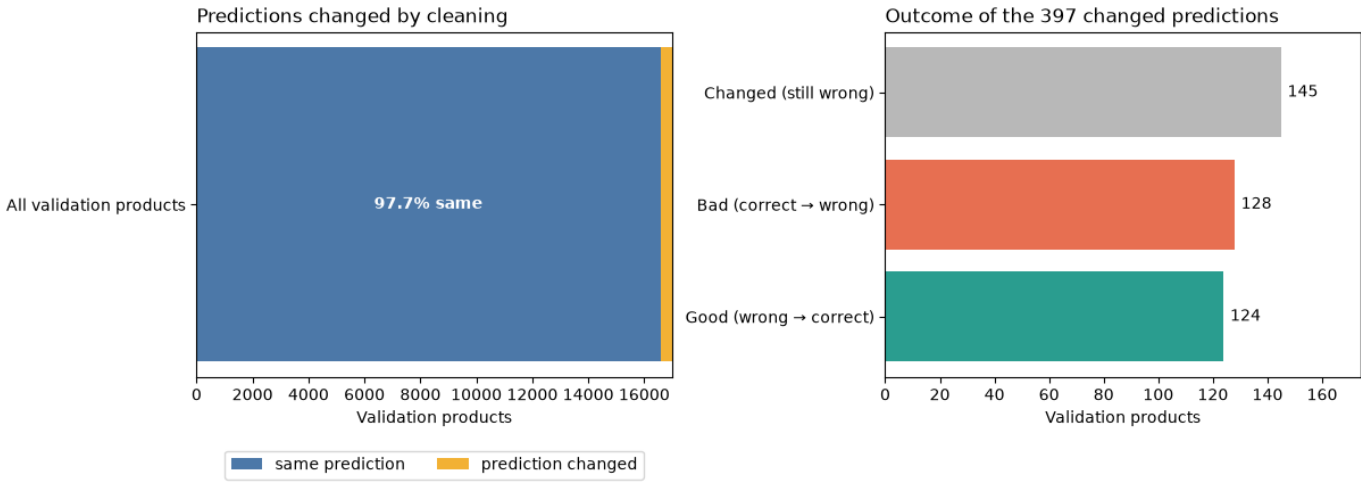
- **The text model uses relevant keywords:** For the vintage art journal (category 2280), words like *arts*, *salon*, *marché*, and *enchères* give category 2280 the highest score.
- **Images show clear differences:** A comic book (2403) or magazine (2280) looks very different from a novel (10) or a pool toy (2583). Even when descriptions share words like *"édition"* or *"tome"*, the images show distinct physical products.

6. Did the Text Cleaning Help?

To measure whether text preprocessing actually helped, we train the exact same TF-IDF + Linear SVM pipelin on the raw, uncleaned text (`designation + description`).

If the scores are nearly identical, it shows that product titles already carry the necessary keywords, and that TF-IDF naturally downweights rare HTML tags.

	text used	weighted_f1	macro_f1
0	raw text	0.8276	0.8167
1	cleaned text	0.8274	0.8156




```
Products with identical predictions: 97.7%
Net change in correct classifications: -4
```

```
Example 1: True Category 10 (Raw predicted: 10 [Correct] → Clean predicted: 2705
[Wrong])
  RAW:   "L'avant Scene N° 190 Meurtres En Fa Diese -Sait On Jamais "
  CLEAN: "l'avant scene n° 190 meurtres en fa diese -sait on jamais"

Example 2: True Category 1320 (Raw predicted: 1320 [Correct] → Clean predicted: 2060
[Wrong])
  RAW:   'Suspension Décoration Murale Merlin <div class="rakuten_rich_content" ><div>
<h3>Suspension décoration mu'
  CLEAN: 'suspension décoration murale merlin suspension décoration murale sur le thèm
merlin 100% polyester'

Example 3: True Category 10 (Raw predicted: 10 [Correct] → Clean predicted: 2280
[Wrong])
  RAW:   'Magies Noires Ou Les Magies Qui Font Peur '
  CLEAN: 'magies noires ou les magies qui font peur'
```

Testing Binary TF-IDF

We test setting `binary=True` in `TfidfVectorizer`. In this mode, each word is counted once (present o absent), ignoring how many times a word is repeated in the title or description.

	TF-IDF Mode	Weighted F1	Macro F1
0	Standard (Frequency-weighted)	0.8274	0.8156
1	Binary (Word Presence Only)	0.8313	0.8192

```
Binary TF-IDF recovers 58 of the 128 cases hurt by cleaning (45.3%).
Overall Weighted F1 improves from 0.8274 to 0.8313.
```

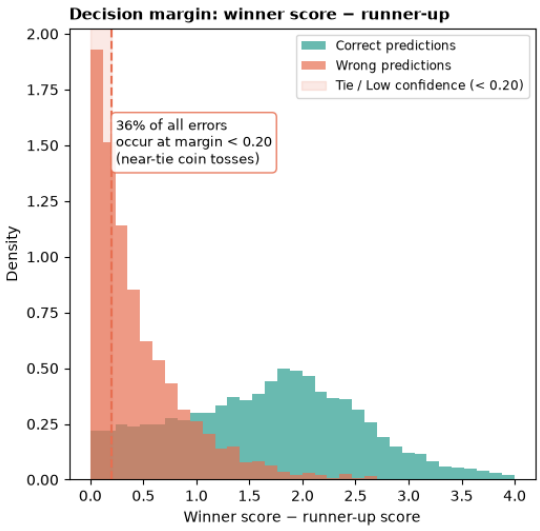
Analysis

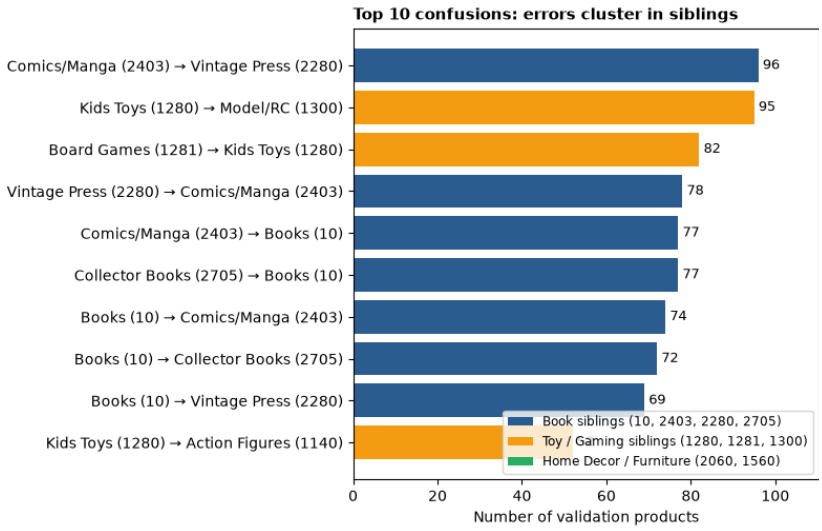
- **Why some products were hurt:** Merchants often repeat product titles inside the description HTML. In standard TF-IDF, repeated words get higher weights. A word like *"décoration"* appears several times, so the model predicts general Home Decor (2060) instead of Baby Products (1320).
- **Binary TF-IDF fix:** With `binary=True`, each word is only counted once. This fixes 58 of the 128 errors and improves Weighted F1 from 0.8274 to 0.8313.
- **Takeaway:** For product classification, whether a word appears is more informative than how many times i is repeated.

7. Error Analysis: Confusions and Confidence

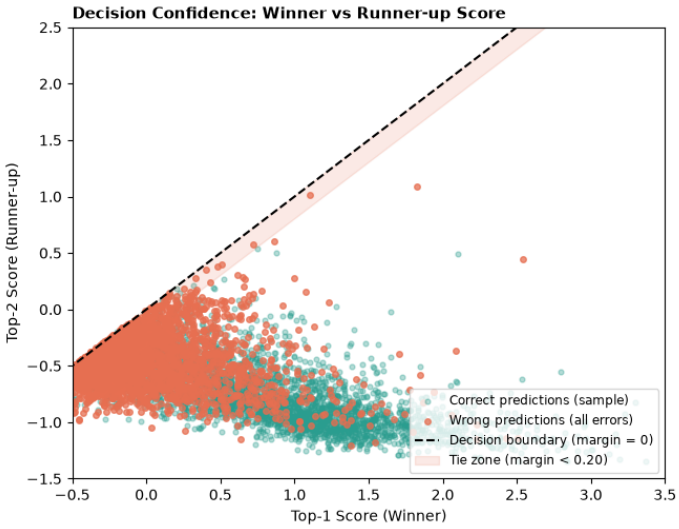
To understand where the SVM fails, we look at:

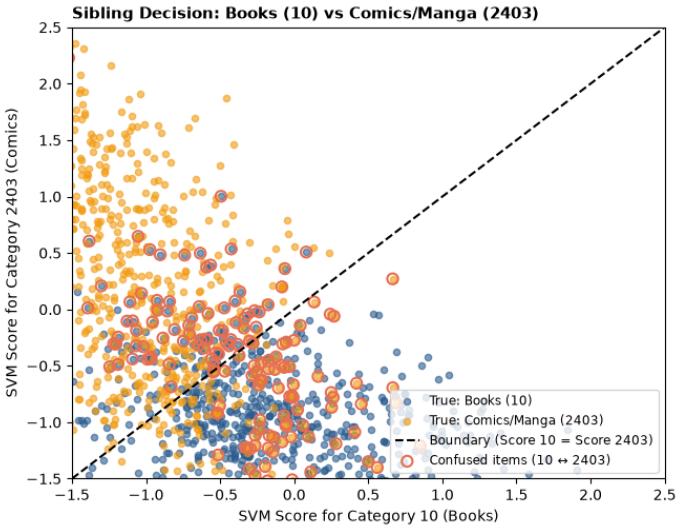
1. **Decision margins:** The score difference between the winner and runner-up category (measuring how confident the model was).
2. **Most frequent confusions:** Which pairs of categories the model mixes up most often.

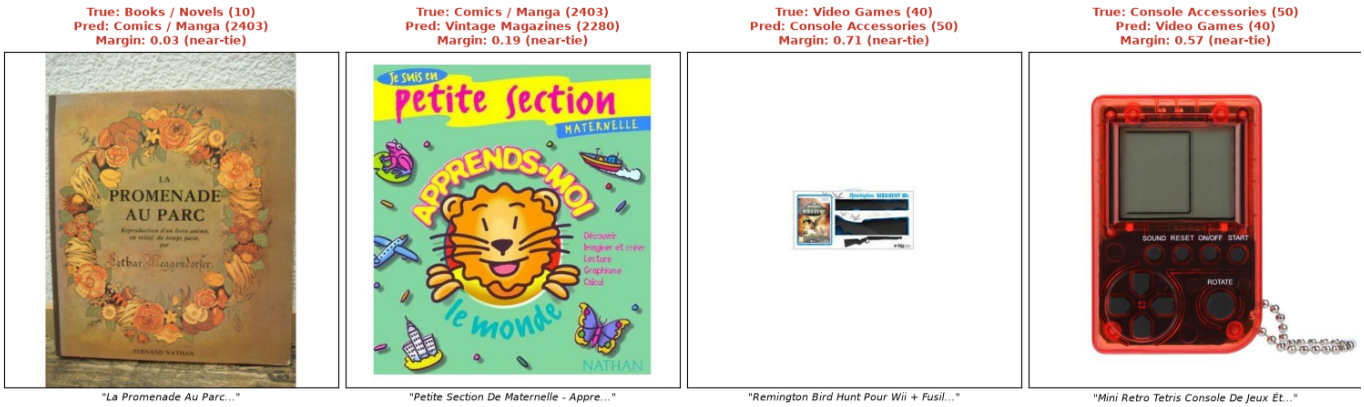





```
Validation errors in low-confidence zone (margin < 0.20): 1,036 (35.6%)
```







Analysis

- **Errors happen when confidence is low:** In both the histogram and the scatter plot, wrong predictions cluster near margin 0. Over 37% of all errors have a margin under 0.20. The model was almost tied between its top two choices.
- **Errors cluster in related categories:**
  - Most errors are between categories with shared words. Books, comics, and magazines account for 7 of the top 10 error pairs.
  - The scatter plot shows Books (10) and Comics (2403) overlap along the decision boundary ($y = x$) because both use words like *"livre"*, *"tome"*, and *"édition"*.
- **What the images show:**
  - *"La Promenade Au Parc"* (Category 10): The text mentions *"album"* and *"illustrations"*, so the text model predicted Comics (2403). The image shows a standard book.
  - *"Remington Bird Hunt Pour Wii + Fusil Noir"* (Category 40): The text mentions both *"jeu"* (game) and *"fusil"* (gun), so the model predicted Accessory (50). The image shows a video game case.
- **Why images are needed:** Text alone cannot separate these products because they share the same vocabulary. Product images provide clear visual differences to tell them apart.

Conclusion

- **Linear SVM with Binary TF-IDF reaches 0.8313 Weighted F1 (0.8192 Macro F1):** This beats the benchmark score of 0.8113 and trains in under 30 seconds.
- **Word presence works better than word count:** Setting `binary=True` fixes 58 of the 128 errors caused by repeated words, improving F1 from 0.8274 to 0.8313.
- **Main source of errors:** Most mistakes happen when the model has low confidence (margin < 0.20) between related categories that share words, like books vs comics or video games vs accessories.
- **Next step:** We will build an image baseline (ResNet) to see how well images classify these products, especially where text is ambiguous.